

CMSC426_Assignment5_sp26

May 14, 2026

1 Assignment: CNN Classification and Segmentation

Name: Arik Gershman

1.1 Overview

In this assignment, you will work with two core computer vision tasks:

1. Image classification using a convolutional neural network trained on CIFAR-10.
2. Image segmentation using the Segment Anything Model, SAM.

Classification setting: - Batch size: 4 - Epochs: 5 - Optimizer: Adam - Learning rate: 0.001

Expected classification accuracy: - A correct implementation should usually reach around 45% to 60% test accuracy with this small batch size and short training time. - Results may vary slightly depending on runtime, hardware, and random initialization.

```
[1]: import os
import sys
import random
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader

seed = 42
random.seed(seed)
np.random.seed(seed)
```

```
torch.manual_seed(seed)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cuda

2 Part 1: Image Classification with CNN (80 pts, including bonus points)

2.1 Goal

In this part, you will train a convolutional neural network on CIFAR-10.

You will:

1. Load CIFAR-10.
2. Display sample training images.
3. Define a CNN.
4. Train the CNN.
5. Evaluate test accuracy.
6. Compute per-class accuracy and average class accuracy.
7. Plot training loss and test accuracy.

2.2 1.1 Load CIFAR-10

```
[2]: !pip install datasets -q
```

```
[3]: from datasets import load_dataset

batch_size = 4

transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                          (0.5, 0.5, 0.5))
])

class HFCIFAR10Dataset(Dataset):
    def __init__(self, split, transform=None):
        self.dataset = load_dataset("uoft-cs/cifar10", split=split)
        self.transform = transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
```

```

        item = self.dataset[idx]
        image = item["img"]
        label = item["label"]

        if self.transform:
            image = self.transform(image)

        return image, label

trainset = HFCIFAR10Dataset("train", transform=transform)
testset = HFCIFAR10Dataset("test", transform=transform)

trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True,
    ↪num_workers=2)
testloader = DataLoader(testset, batch_size=batch_size, shuffle=False,
    ↪num_workers=2)

classes = ("plane", "car", "bird", "cat", "deer",
           "dog", "frog", "horse", "ship", "truck")

print("Training images:", len(trainset))
print("Testing images:", len(testset))
print("Batch size:", batch_size)

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93:

UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

README.md: 0.00B [00:00, ?B/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

plain_text/train-00000-of-00001.parquet: 0%| | 0.00/120M [00:00<?, ?B/ ↪s]

plain_text/test-00000-of-00001.parquet: 0%| | 0.00/23.9M [00:00<?, ?B/ ↪s]

Generating train split: 0%| | 0/50000 [00:00<?, ? examples/s]

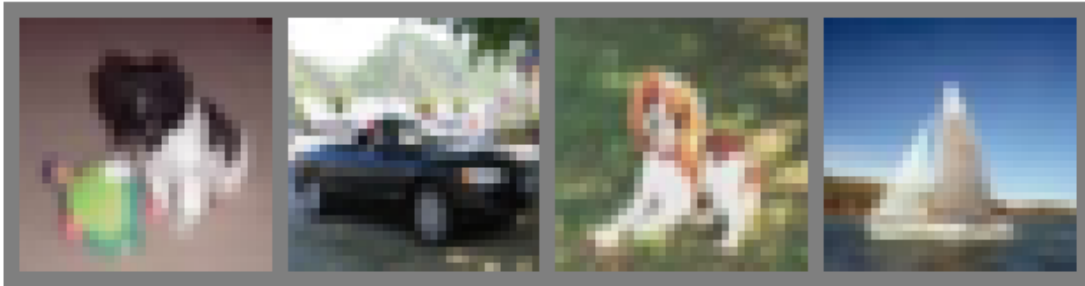
```
Generating test split: 0%|          | 0/10000 [00:00<?, ? examples/s]
Training images: 50000
Testing images: 10000
Batch size: 4
```

2.3 1.2 Display Training Images

```
[4]: def imshow(img):
      img = img / 2 + 0.5
      npimg = img.numpy()
      plt.figure(figsize=(8, 4))
      plt.imshow(np.transpose(npimg, (1, 2, 0)))
      plt.axis("off")
      plt.show()

      dataiter = iter(trainloader)
      images, labels = next(dataiter)

      imshow(torchvision.utils.make_grid(images))
      print("Labels:", " ".join(classes[labels[j]] for j in range(len(labels))))
```



Labels: dog car dog ship

2.4 1.3 Define a Simple CNN (20 pts)

The target architecture is:

```
Conv -> ReLU -> MaxPool
Conv -> ReLU -> MaxPool
Flatten
Linear -> ReLU -> Linear
```

This is a lightweight CNN designed to run faster in Colab.

```
[81]: class Net(nn.Module):
      def __init__(self):
          super().__init__()
```

```

        """
        Add layers to your neural net.
        Tutorial for reference: https://pytorch.org/tutorials/beginner/introyt/trainingyt.html
        """

        # TODO:
        # Define two convolution layers.
        # The first should take RGB images as input.
        self.conv1 = nn.Conv2d(3, 32, 5)
        self.conv2 = nn.Conv2d(32, 64, 5)

        # TODO:
        # Define a max pooling layer.
        self.pool = nn.MaxPool2d(2, 2)

        # TODO:
        # Define two fully connected layers.
        self.fc1 = nn.Linear(64 * 5 * 5, 256)
        self.fc2 = nn.Linear(256, 128)

        # Define another fully connected layer.
        self.fc3 = nn.Linear(128, 10)

    def forward(self, x):
        """
        Forward pass:
        Apply layers you defined in __init__() to RGB input
        layers: Conv2d --> ReLU --> MaxPool2d --> Conv2d --> ReLU -->
        ↪MaxPool2d --> Flatten --> Linear --> ReLU --> Linear --> ReLU --> Linear
        """
        # TODO:
        # Apply conv -> relu -> pool twice.
        # Flatten the tensor.
        # Apply two fully connected layers.
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 64 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net().to(device)
print(net)

```

Net(

```

(conv1): Conv2d(3, 32, kernel_size=(5, 5), stride=(1, 1))
(conv2): Conv2d(32, 64, kernel_size=(5, 5), stride=(1, 1))
(pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
(fc1): Linear(in_features=1600, out_features=256, bias=True)
(fc2): Linear(in_features=256, out_features=128, bias=True)
(fc3): Linear(in_features=128, out_features=10, bias=True)
)

```

2.5 1.4 Define Loss and Optimizer (5 pts)

Use cross-entropy loss and Adam optimizer.

The learning rate is fixed at 0.001.

```

[82]: # TODO YOUR CODE HERE: Add loss function (hint: use nn.function_name())
      criterion = nn.CrossEntropyLoss()

      # TODO YOUR CODE HERE: Add optimizer (hint: use Adam)
      optimizer = optim.Adam(net.parameters(), lr=0.001)

```

2.6 1.5 Train the Network (10 pts)

Train for 5 epochs. During training, record:

- average training loss per epoch
- test accuracy per epoch
- You should at least get 60% accuracy

2.7 Extra Credit (5 pts)

- 5 pts bonus if you get more than 65% accuracy

```

[83]: epochs = 5

train_losses = []
test_accuracies = []

for epoch in range(epochs):
    net.train()
    running_loss = 0.0
    num_batches = 0

    for images, labels in trainloader:
        images = images.to(device)
        labels = labels.to(device)

        # TODO:
        # 1. Clear gradients.

```

```

optimizer.zero_grad()
# 2. Run forward pass.
outputs = net(images)
# 3. Compute loss.
loss = criterion(outputs, labels)
# 4. Run backward pass.
loss.backward()
# 5. Update weights.
optimizer.step()

running_loss += loss.item()
num_batches += 1

avg_loss = running_loss / num_batches
train_losses.append(avg_loss)

net.eval()
correct = 0
total = 0

with torch.no_grad():
    for images, labels in testloader:
        images = images.to(device)
        labels = labels.to(device)

        # TODO:
        # Run prediction and update correct / total.
        preds = net(images)
        _, predicted = torch.max(preds.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

test_acc = 100 * correct / total
test accuracies.append(test_acc)

print(f"Epoch {epoch + 1:02d}/{epochs}, Loss: {avg_loss:.4f}, Test Accuracy:
↪ {test_acc:.2f}%")

```

```

Epoch 01/5, Loss: 1.4697, Test Accuracy: 55.33%
Epoch 02/5, Loss: 1.1573, Test Accuracy: 61.10%
Epoch 03/5, Loss: 1.0448, Test Accuracy: 62.87%
Epoch 04/5, Loss: 0.9743, Test Accuracy: 64.54%
Epoch 05/5, Loss: 0.9254, Test Accuracy: 66.53%

```

2.8 1.6 Final Per-Class Accuracy and Average Class Accuracy (10 pts)

Compute accuracy for each CIFAR-10 class.

Then compute average class accuracy:

```
average_class_accuracy = mean(per-class accuracies)
```

```
[84]: correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

net.eval()

with torch.no_grad():
    for images, labels in testloader:
        images = images.to(device)
        labels = labels.to(device)

        # TODO:
        # 1. Compute model outputs.
        outputs = net(images)
        # 2. Convert outputs into predicted labels.
        _, predictions = torch.max(outputs, 1)
        # 3. Update correct_pred and total_pred for each class.
        for label, prediction in zip(labels, predictions):
            label_idx = label.item()
            pred_idx = prediction.item()
            classname = classes[label_idx]
            if label_idx == pred_idx:
                correct_pred[classname] += 1
            total_pred[classname] += 1

class_accuracies = []

for classname in classes:
    accuracy = 100 * correct_pred[classname] / total_pred[classname]
    class_accuracies.append(accuracy)
    print(f"Accuracy for class {classname:5s}: {accuracy:.1f}%")

# TODO:
# Compute average_class_accuracy as the mean of class_accuracies.
average_class_accuracy = sum(class_accuracies) / len(class_accuracies)

print(f"Average class accuracy: {average_class_accuracy:.1f}%")
```

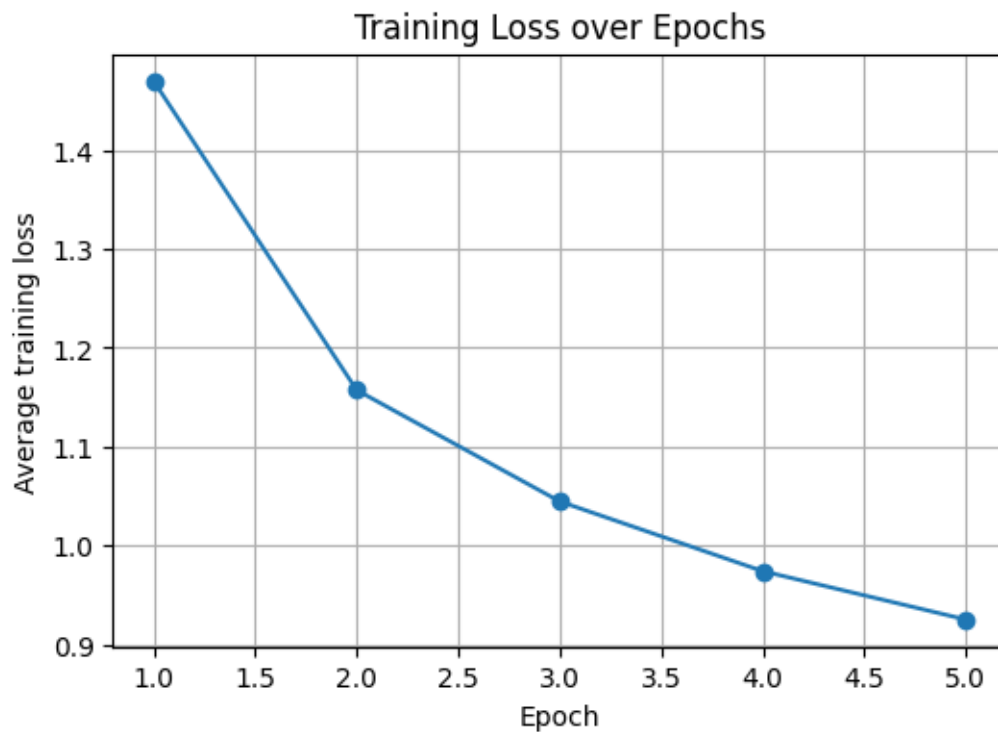
```
Accuracy for class plane: 59.2%
Accuracy for class car : 83.3%
Accuracy for class bird : 49.3%
Accuracy for class cat : 49.5%
Accuracy for class deer : 70.1%
Accuracy for class dog : 61.3%
Accuracy for class frog : 68.5%
```

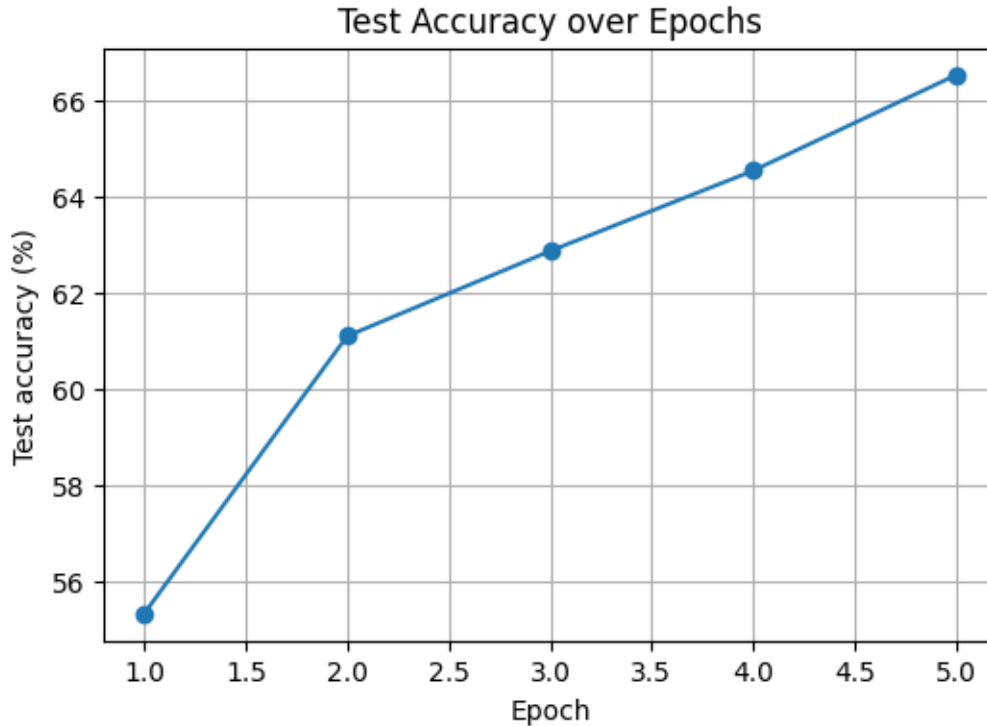
Accuracy for class horse: 65.7%
Accuracy for class ship : 84.6%
Accuracy for class truck: 73.8%
Average class accuracy: 66.5%

2.9 1.7 Plots (5 pts)

```
[85]: plt.figure(figsize=(6, 4))
plt.plot(range(1, len(train_losses) + 1), train_losses, marker="o")
plt.xlabel("Epoch")
plt.ylabel("Average training loss")
plt.title("Training Loss over Epochs")
plt.grid(True)
plt.show()

plt.figure(figsize=(6, 4))
plt.plot(range(1, len(test_accuracies) + 1), test_accuracies, marker="o")
plt.xlabel("Epoch")
plt.ylabel("Test accuracy (%)")
plt.title("Test Accuracy over Epochs")
plt.grid(True)
plt.show()
```





2.10 Part 1 Write-up (20 pts)

Answer the following questions.

1. What is your final test accuracy? 66.53%
2. What is your average class accuracy? 66.5%
3. Which CIFAR-10 classes are easiest or hardest for your model? Easiest: ship, car, and truck. Hardest: bird, cat, and plane.
4. Why might average class accuracy be useful in addition to overall test accuracy? Average class accuracy is helpful especially in situations of class imbalance. If one class highly dominates the dataset, the network may generalize to simply predict that class every time. The overall test accuracy will lead us to conclude that the model does well even though it doesn't generalize at all. Assuming there are only 2 classes, the average class accuracy will show that the model doesn't do well by averaging the accuracy for class 1 (~100%) and the accuracy for class 2 (~0%), for an average of about 50%. Using the average class accuracy in addition to the overall test accuracy will may still prove helpful when there are more than 2 classes, and when the imbalance isn't so extreme but still heavily skewed.

2.11 Extra Credits (5 pt)

Run VGG with pre-trained weights in this [colab](#). Test any two images of your choice to your model and to VGG model and show accuracy (images must include objects from CIFAR10 classes). Discuss which model performs better and why.

```
[54]: model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg11', pretrained=True)
# or any of these variants
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg11_bn', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg13', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg13_bn', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg16', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg16_bn', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg19', pretrained=True)
# model = torch.hub.load('pytorch/vision:v0.10.0', 'vgg19_bn', pretrained=True)
model.eval()
net.eval()
model = model.to(device)
```

Downloading: "https://github.com/pytorch/vision/zipball/v0.10.0" to
/root/.cache/torch/hub/v0.10.0.zip

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208:
UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be
removed in the future, please use 'weights' instead.
warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223:
UserWarning: Arguments other than a weight enum or `None` for 'weights' are
deprecated since 0.13 and may be removed in the future. The current behavior is
equivalent to passing `weights=VGG11\_Weights.IMAGENET1K\_V1`. You can also use
`weights=VGG11\_Weights.DEFAULT` to get the most up-to-date weights.
warnings.warn(msg)

Downloading: "https://download.pytorch.org/models/vgg11-8a719046.pth" to
/root/.cache/torch/hub/checkpoints/vgg11-8a719046.pth

100%| | 507M/507M [00:05<00:00, 95.1MB/s]

```
[55]: # Download ImageNet labels
!wget https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt
```

--2026-05-14 03:38:07--

https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt

Resolving raw.githubusercontent.com (raw.githubusercontent.com)...

185.199.111.133, 185.199.109.133, 185.199.108.133, ...

Connecting to raw.githubusercontent.com

(raw.githubusercontent.com)|185.199.111.133|:443... connected.

HTTP request sent, awaiting response... 200 OK

Length: 10472 (10K) [text/plain]

Saving to: 'imagenet_classes.txt'

imagenet_classes.tx 0%[] 0 --.-KB/s

imagenet_classes.tx 100%[=====>] 10.23K --.-KB/s in 0s

2026-05-14 03:38:07 (105 MB/s) - 'imagenet_classes.txt' saved [10472/10472]

```

[56]: # Read the categories
with open("imagenet_classes.txt", "r") as f:
    categories = [s.strip() for s in f.readlines()]

[80]: image_paths = [
        "/content/my_image_1.jpg",
        "/content/my_image_2.png"
    ]

    preprocess = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])

    indices_to_test = [2, 17]
    img_paths = ["airplane.jpg", "frog.jpg", "bird.jpg"]

    for img_path in img_paths:
        raw_image = Image.open(img_path).convert('RGB')

        print(f"\nTesting image: {img_path}")

        # Display image for reference
        plt.figure(figsize=(2, 2))
        plt.imshow(raw_image)
        plt.axis("off")
        plt.show()

        # VGG
        input = preprocess(raw_image).unsqueeze(0).to(device)

        if torch.cuda.is_available():
            input = input.to('cuda')
            model.to('cuda')

        with torch.no_grad():
            output = model(input)

        probabilities = torch.nn.functional.softmax(output[0], dim=0)
        top1_prob, top1_catid = torch.topk(probabilities, 1)
        print("VGG prediction:")
        print(categories[top1_catid[0]], top1_prob[0].item())

```

```

# My model
input = transform(raw_image).unsqueeze(0).to(device)

if torch.cuda.is_available():
    input = input.to('cuda')
    net.to('cuda')

with torch.no_grad():
    preds = net(input)
    my_probabilities = torch.nn.functional.softmax(preds[0], dim=0)
    my_top1_prob, my_top1_catid = torch.topk(my_probabilities, 1)

    predicted_class_name = classes[my_top1_catid[0].item()]
    predicted_prob = my_top1_prob[0].item()

    print("My Model prediction:")
    print(f"{predicted_class_name} {predicted_prob}")

```

Testing image: airplane.jpg



VGG prediction:
 airliner 0.9371960759162903
 My Model prediction:
 plane 0.9999992847442627

Testing image: frog.jpg



```
VGG prediction:
bullfrog 0.6276424527168274
My Model prediction:
bird 0.34306979179382324
```

Testing image: bird.jpg



```
VGG prediction:
jacamar 0.24331825971603394
My Model prediction:
bird 0.9999998807907104
```

It seems that VGG did better than my simple model when given the 3 images above. The VGG model has 1000 prediction classes, so it can get much more specific about its prediction. The VGG model predicted all 3 images correctly (broadly speaking. I didn't verify that the frog in the image is a bullfrog and the bird is a jacamar, but as long as it got the right idea). My simple model only got 2 out of these 3 right, which makes sense given the ~67% accuracy it achieved on the test set above. The VGG model is pre-trained on a large dataset and is a much stronger and more robust model with millions of parameters. It is able to generalize very well. I believe that's the reason the VGG model did better than my model.

3 Part 2: Image Segmentation with SAM (30 pts)

3.1 Goal

In this part, you will use the Segment Anything Model, SAM, for image segmentation.

You will:

1. Load SAM.
2. Run SAM on a provided example image.
3. Run SAM on two images of your choice.
4. Analyze one failure case.

To make this assignment easier to run, we use the smaller `vit_b` SAM model.

```
[15]: %%capture
      !pip install opencv-python matplotlib
      !pip install 'git+https://github.com/facebookresearch/segment-anything.git'
```

```

!mkdir -p images
!wget -q -P images https://raw.githubusercontent.com/facebookresearch/
↳segment-anything/main/notebooks/images/truck.jpg
!wget -q -P images https://raw.githubusercontent.com/facebookresearch/
↳segment-anything/main/notebooks/images/groceries.jpg

!wget -q https://dl.fbaipublicfiles.com/segment_anything/sam_vit_b_01ec64.pth

```

```

[16]: import cv2
from PIL import Image

def show_mask(mask, ax, random_color=False):
    if random_color:
        color = np.concatenate([np.random.random(3), np.array([0.6])], axis=0)
    else:
        color = np.array([30 / 255, 144 / 255, 255 / 255, 0.6])

    h, w = mask.shape[-2:]
    mask_image = mask.reshape(h, w, 1) * color.reshape(1, 1, -1)
    ax.imshow(mask_image)

def show_points(coords, labels, ax, marker_size=375):
    pos_points = coords[labels == 1]
    neg_points = coords[labels == 0]

    ax.scatter(pos_points[:, 0], pos_points[:, 1],
               color="green", marker="*", s=marker_size,
               edgecolor="white", linewidth=1.25)

    ax.scatter(neg_points[:, 0], neg_points[:, 1],
               color="red", marker="*", s=marker_size,
               edgecolor="white", linewidth=1.25)

def load_rgb_image(path):
    image = cv2.imread(path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    return image

```

```

[17]: from segment_anything import sam_model_registry, SamPredictor

sam_checkpoint = "sam_vit_b_01ec64.pth"
model_type = "vit_b"

sam = sam_model_registry[model_type](checkpoint=sam_checkpoint)
sam.to(device=device)

```

```
predictor = SamPredictor(sam)
print("SAM loaded.")
```

SAM loaded.

3.2 2.1 Example: Segment an Object in the Truck Image

```
[22]: image = load_rgb_image("images/truck.jpg")
      predictor.set_image(image)

      input_point = np.array([[500, 375]])
      input_label = np.array([1])

      masks, scores, logits = predictor.predict(
          point_coords=input_point,
          point_labels=input_label,
          multimask_output=True,
      )

      best_mask_index = np.argmax(scores)

      plt.figure(figsize=(8, 8))
      plt.imshow(image)
      show_mask(masks[best_mask_index], plt.gca())
      show_points(input_point, input_label, plt.gca())
      plt.title(f"Best mask, score = {scores[best_mask_index]:.3f}")
      plt.axis("off")
      plt.show()
```

Best mask, score = 0.984



3.3 2.2 Your Own Images (10 pts)

Run SAM on two images of your choice.

For each image:

1. Display the original image.
2. Choose one foreground point.
3. Display one segmentation mask.
4. Briefly explain whether the mask is good or bad.

```
[38]: # TODO:
# Replace the image paths and input points below with your own choices.
# You may upload images to Colab and use their file paths.

image_path_1 = "lion.jpg"
input_point_1 = np.array([[2500, 2000]])

image_path_2 = "testudo.jpg"
input_point_2 = np.array([[530, 190]])

for path, point in [(image_path_1, input_point_1), (image_path_2,
↪input_point_2)]:
```

```

image = load_rgb_image(path)
predictor.set_image(image)

# Show original image
plt.figure(figsize=(8, 8))
plt.axis("off")
plt.imshow(image)

input_point = point
input_label = np.array([1])

# TODO:
# Run predictor.predict.
masks, scores, logits = predictor.predict(
    point_coords=input_point,
    point_labels=input_label,
    multimask_output=True,
)
# Select one mask.
best_mask_index = np.argmax(scores)
# Display the image with the selected mask and point.
plt.figure(figsize=(8, 8))
plt.imshow(image)
show_mask(masks[best_mask_index], plt.gca())
show_points(input_point, input_label, plt.gca())
plt.title(f"Best mask, score = {scores[best_mask_index]:.3f}")
plt.axis("off")
plt.show()

```



Best mask, score = 0.955





Best mask, score = 0.936



3.4 2.3 Failure Case (5 pts)

Find one case where SAM does not segment the object well.

Examples: - The mask includes too much background. - The mask misses part of the object. - The object is very small. - The object is occluded. - The scene has many overlapping objects.

```
[47]: # TODO:  
      # Run SAM on one failure case.  
      # Display the image, selected point, and mask.
```

```

# Then explain why the result may have failed.

image = load_rgb_image("cat.jpg")
predictor.set_image(image)

# Show original image
plt.figure(figsize=(8, 8))
plt.axis("off")
plt.imshow(image)

input_point = np.array([[400, 300]])
input_label = np.array([1])

masks, scores, logits = predictor.predict(
    point_coords=input_point,
    point_labels=input_label,
    multimask_output=True,
)

best_mask_index = np.argmax(scores)

plt.figure(figsize=(8, 8))
plt.imshow(image)
show_mask(masks[best_mask_index], plt.gca())
show_points(input_point, input_label, plt.gca())
plt.title(f"Best mask, score = {scores[best_mask_index]:.3f}")
plt.axis("off")
plt.show()

"""
The above cat picture is a case where SAM fails successfully segment the object.
The human eye slightly struggles to see the full bounds of the cat, but it is
still pretty clear. SAM misses some parts of the cat and includes parts of the
blanket. I believe the reason this result has failed is because the foreground
and background of the image are both white, and it is difficult for SAM to
distinguish between them. I also think it did not segment the cat's eyes
because they seem like they are a part of a different object due to their
contrast with the rest of the cat's body.
"""

```



Best mask, score = 0.960



3.5 Part 2 Write-up (15 pts)

Answer the following questions.

1. What types of images or objects does SAM segment well? SAM successfully segments objects that have clear outlines, which are objects that have a strong contrast between the foreground and the background. Also, images that have distinct and no overlapping objects. Some examples of such images include the lion and Testudo images I used above.
2. What types of images or objects are difficult for SAM? SAM struggles with the opposite. It struggles with images/objects that do not have strong contrast between the foreground and the background. Meaning, images where it is unclear where an object's edge/outline is. It also struggles with images that have overlapping or obscured objects. The above cat image is an example of an image SAM struggles with.
3. How does changing the input point affect the result? Changing the input point changes SAM's focus within the image, segmenting different objects. For example, if there are 2 distinct objects in an image, placing the input point on one of them will have SAM create the best mask for that object, whereas placing it on the other one will segment the other one.

4 Grading Rubric

Accuracy guidance for Part 1: - Below 45%: likely implementation or training issue - 45% to 55%: acceptable for this small model and short training setting - 55% to 60%: good result - Above 60%: strong result for this lightweight CNN

5 Save as PDF

```
[86]: !apt-get update
      !dpkg --configure -a
      !apt-get install -y texlive texlive-xetex texlive-latex-extra pandoc
      !pip install pypandoc
```

```
0% [Working]          Get:1 https://cloud.r-project.org/bin/linux/ubuntu
jammy-cran40/ InRelease [3,632 B]
0% [Waiting for headers] [Waiting for headers] [1 InRelease 0 B/3,632 B 0%] [Co
0% [Waiting for headers] [Waiting for headers] [Connected to r2u.stat.illinois.
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
0% [Waiting for headers] [Waiting for headers] [Connected to r2u.stat.illinois.
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
0% [Waiting for headers] [3 InRelease 9,773 B/129 kB 8%] [Connected to r2u.stat
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
0% [Waiting for headers] [3 InRelease 18.5 kB/129 kB 14%] [Connected to r2u.sta
Get:5 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
InRelease [1,581 B]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:7 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Get:8 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
```

```

Get:9 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
Packages [2,617 kB]
Hit:10 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Get:11 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[3,915 kB]
Hit:12 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy
InRelease
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages
[1,602 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[1,295 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,247
kB]
Hit:16 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:17 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [3,007 kB]
Get:18 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [10.2 MB]
Fetched 27.3 MB in 2s (11.4 MB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
pandoc set to manually installed.
The following additional packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
  libsyntaxtex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
  lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
  ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
  teckit tex-common tex-gyre texlive-base texlive-binaries
  texlive-fonts-recommended texlive-latex-base texlive-latex-recommended
  texlive-pictures texlive-plain-generic tipa xfonts-encodings xfonts-utils
Suggested packages:
  fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
  libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
  poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
  fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
  fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
  | postscript-viewer perl-tk xpdf | pdf-viewer xzdec
  texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
  icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
  texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
  texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex

```

default-jre-headless tipa-doc

The following NEW packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration tiutils
teckit tex-common tex-gyre texlive texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils

0 upgraded, 54 newly installed, 0 to remove and 51 not upgraded.

Need to get 182 MB of archives.

After this operation, 571 MB of additional disk space will be used.

Get:1 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-droid-fallback all 1:6.0.1r16-1.1build1 [1,805 kB]

Get:2 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-lato all 2.0-2.1 [2,696 kB]

Get:3 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 poppler-data all 0.4.11-1 [2,171 kB]

Get:4 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-common all 6.17 [33.7 kB]

Get:5 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-urw-base35 all 20200910-1 [6,367 kB]

Get:6 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9-common all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]

Get:7 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libidn12 amd64 1.38-4ubuntu1 [60.0 kB]

Get:8 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libijs-0.35 amd64 0.35-15build2 [16.5 kB]

Get:9 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libjbig2dec0 amd64 0.19-3build2 [64.7 kB]

Get:10 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9 amd64 9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]

Get:11 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libkpathsea6 amd64 2021.20210626.59705-1ubuntu0.3 [60.6 kB]

Get:12 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libwoff1 amd64 1.0.2-1build4 [45.2 kB]

Get:13 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 dvisvgm amd64 2.13.1-1 [1,221 kB]

Get:14 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-lmodern all 2.004.5-6.1 [4,532 kB]

Get:15 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-noto-mono all 20201225-1build1 [397 kB]

Get:16 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 fonts-texgyre all 20180621-3.1 [10.2 MB]

Get:17 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libapache-pom-java
all 18-1 [4,720 B]
Get:18 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-parent-
java all 43-1 [10.8 kB]
Get:19 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-logging-
java all 1.2-2 [60.3 kB]
Get:20 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libptexenc1
amd64 2021.20210626.59705-1ubuntu0.3 [39.1 kB]
Get:21 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rubygems-integration
all 1.18 [5,336 B]
Get:22 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64
3.0.2-7ubuntu2.12 [50.1 kB]
Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems
all 3.3.5-2ubuntu1.2 [228 kB]
Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1
[5,100 B]
Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7
kB]
Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all
0.1.1-2 [12.6 kB]
Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick
all 1.7.0-3ubuntu0.2 [52.5 kB]
Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all
0.3.2-1ubuntu0.1 [24.9 kB]
Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0
amd64 3.0.2-7ubuntu2.12 [5,113 kB]
Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsinctex2
amd64 2021.20210626.59705-1ubuntu0.3 [55.8 kB]
Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64
2.5.11+ds1-1 [421 kB]
Get:32 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53
amd64 2021.20210626.59705-1ubuntu0.3 [120 kB]
Get:33 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexluajit2
amd64 2021.20210626.59705-1ubuntu0.3 [267 kB]
Get:34 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libzip-0-13 amd64
0.13.72+dfsg.1-1.1 [27.0 kB]
Get:35 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-encodings all
1:1.0.5-0ubuntu2 [578 kB]
Get:36 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-utils amd64
1:7.7+6build2 [94.6 kB]
Get:37 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 lmodern all
2.004.5-6.1 [9,471 kB]
Get:38 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 preview-latex-style
all 12.2-1ubuntu1 [185 kB]
Get:39 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 t1utils amd64
1.41-4build2 [61.3 kB]
Get:40 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 teckit amd64
2.5.11+ds1-1 [699 kB]

```

Get:41 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-gyre all
20180621-3.1 [6,209 kB]
Get:42 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 texlive-
binaries amd64 2021.20210626.59705-1ubuntu0.3 [9,861 kB]
Get:43 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-base all
2021.20220204-1 [21.0 MB]
Get:44 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-fonts-
recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base
all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all
2021.20220204-1 [14.3 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 2s (79.0 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 122412 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...

```

```

Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package rubygems-integration.

```

```

Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libtexluaajit2:amd64.
Preparing to unpack
.../32-libtexluaajit2_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../33-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.

```

```

Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package tlutils.
Preparing to unpack .../38-tlutils_1.41-4build2_amd64.deb ...
Unpacking tlutils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../45-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive.
Preparing to unpack .../46-texlive_2021.20220204-1_all.deb ...
Unpacking texlive (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../47-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...
Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../48-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../49-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../50-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...

```

```

Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../51-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../52-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../53-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
update-language: texlive-base not installed and configured, doing nothing!
Setting up libbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsynchronet2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin

```

```

(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive (2021.20220204-1) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.12) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.8) ...
/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

```

```

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

Processing triggers for tex-common (6.17) ...
Running updmmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time... done.
Collecting py pandoc
  Downloading py pandoc-1.17-py3-none-any.whl.metadata (18 kB)
  Downloading py pandoc-1.17-py3-none-any.whl (23 kB)
Installing collected packages: py pandoc
Successfully installed py pandoc-1.17

```

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

```
[ ]: !cp /content/drive/MyDrive/CMSC426/CMSC426_Assignment5_sp26.ipynb ./
!jupyter nbconvert --to PDF "CMSC426_Assignment5_sp26.ipynb"
```